
LoRA-Prop: Time-Warped Low-Rank Feature Propagation for Extreme Low-Step Diffusion

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We address a central bottleneck in accelerating diffusion models: in the extreme
2 low-step regime (three to eight function evaluations), coarse ODE solvers incur
3 truncation and stability errors while cache-based encoder reuse falters because
4 consecutive states are no longer similar. We propose LoRA-Prop with Time-
5 Warp, a plug-in that preserves base checkpoints and avoids heavy teacher-student
6 distillation. Time-Warp discovers information-rich key steps by a dry pass that
7 measures per-layer feature changes, and invokes the full UNet only at these steps.
8 At non-key steps, LoRA-Prop linearly extrapolates intermediate features via tiny,
9 time-conditioned low-rank adapters, enabling a partial forward that is about ten
10 percent of the UNet FLOPs. Adapters are trained once on a small corpus, consti-
11 tute less than one percent parameters, can run in INT8, and are model-agnostic.
12 We detail a reproducible evaluation on SD-1.5 and DiT-XL/2 with COCO and
13 ImageNet, including statistical testing, ablations (rank, key-step schedules, INT8),
14 FLOP accounting, and mobile deployment. A toy run exercises the pipeline end-to-
15 end: LoRA-Prop improves throughput over the full model but matches cache-only
16 MSE, motivating full-scale experiments to reveal benefits under realistic low-step
17 conditions.

18 1 Introduction

19 Diffusion models set the standard for controllable image generation, but their iterative sampling
20 forces dozens of heavy network evaluations per image. Modern accelerators follow two main paths.
21 First, fast samplers discretize the probability flow ODE with higher-order methods or optimized grids,
22 sharply reducing the number of function evaluations (NFEs) Zhou et al. [2023], Xue et al. [2024],
23 Wizadwongsa et al. [2023], Wizadwongsa and Suwajanakorn [2023], *aut.* Second, systems techniques
24 shrink per-step cost by reusing encoder activations, early exiting submodules, or quantizing operators
25 Li et al. [2023a], Moon et al. [2024], Shang et al. [2022], So et al. [2023]. Distillation compresses
26 many-step teachers into few-step students Meng et al. [2022], Salimans et al. [2024], Hsiao et al.
27 [2024], Kim et al. [2024]. Despite substantial progress, all families face a shared hard case: when
28 the step budget is extremely small (three to eight), numerical truncation and stability issues degrade
29 quality, while feature similarity across adjacent steps collapses, breaking encoder reuse and naive
30 early exiting.

31 Our target regime is exactly this hard case. The challenge is dual: retain image quality with sparse
32 solver calls and simultaneously reduce the cost of each call, all without retraining the base model
33 or relying on heavy teacher-student pipelines. Two observations motivate our approach. First,
34 representational dynamics are temporally non-uniform; early and late stages often carry larger feature
35 changes than mid-range steps. Second, when steps are far apart, cached features are stale, but a
36 learned, time-aware extrapolator may predict intermediate representations accurately enough to avoid
37 recomputing expensive blocks.

We introduce LoRA-Prop with Time-Warp, a simple, practical plug-in with three components. Time-Warp runs a single dry pass and selects K key steps that maximize aggregate per-layer feature change; the full UNet is evaluated only at these instants. For non-key steps, LoRA-Prop freezes the base network and attaches small rank- r adapters that extrapolate features from the nearest previous key time using a time-gap embedding. A hybrid sampler executes on the warped grid, calling the full model at key steps and a light non-key path elsewhere. The design preserves base checkpoints and fine-tunes, requires only a once-trained set of adapters (less than one percent parameters), and supports INT8 execution, making it compatible with existing ecosystems.

We position our work relative to prior art. Solver improvements reduce NFEs but remain vulnerable at extreme sparsity Zhou et al. [2023], Xue et al. [2024], Wizardwongsa et al. [2023], aut. Encoder propagation and early exiting reduce per-step work but assume small inter-step drift that does not hold when NFEs shrink to three to eight Li et al. [2023a], Moon et al. [2024]. Distillation attains few-step sampling at substantial training cost and with new student checkpoints that may not remain drop-in compatible Meng et al. [2022], Salimans et al. [2024], Hsiao et al. [2024], Kim et al. [2024]. We instead keep the original model intact, add tiny adapters, and couple where to spend full compute (key steps) with how to avoid recomputation elsewhere (extrapolation).

We verify feasibility through a rigorous, reproducible evaluation plan spanning quality, mechanisms, and systems. Experiment 1 compares FID, IS, CLIPScore, and ImageReward across NFEs on COCO and ImageNet against fast solvers, encoder propagation, adaptive step skipping, and distilled students Zhou et al. [2023], Xue et al. [2024], Wizardwongsa and Suwajanakorn [2023], Li et al. [2023a], Ye et al. [2024], Salimans et al. [2024], Meng et al. [2022], Hsiao et al. [2024]. Experiment 2 analyzes mechanisms via feature-change heatmaps, per-layer prediction errors, and ablations on rank, INT8, and key schedules. Experiment 3 provides FLOP accounting, throughput, and iPhone deployment with ONNX/CoreML parity Li et al. [2023b], Shang et al. [2022], So et al. [2023]. We also report a toy run that exercises the pipeline end-to-end: LoRA-Prop improves throughput relative to the full model but, on this surrogate, does not reduce MSE compared to cache-only reuse. This underscores the need for full-scale evaluations to expose the intended advantage at sparse steps.

1.1 Contributions

- **Unified step and FLOP reduction:** A plug-in accelerator that unifies step reduction and per-step FLOP savings by calling the full UNet only at Time-Warp-discovered key steps and extrapolating features at non-key steps with low-rank adapters.
- **Tiny, compatible adapters:** A once-trained, tiny-parameter adapter scheme that preserves base weights, supports INT8, and is model-agnostic (UNet, DiT, SDXL).
- **Statistical and systems protocol:** A statistically grounded protocol with bootstrapped confidence intervals, paired tests, and robustness analysis across guidance scales, plus comprehensive ablations and systems validation.
- **Public toy validation:** A public toy run validating end-to-end executability and exposing where extrapolation benefits should appear in realistic settings.

Beyond immediate acceleration, our work conceptually links numerical solver design and learned feature prediction. It complements solver-based advances Zhou et al. [2023], Xue et al. [2024], Wizardwongsa and Suwajanakorn [2023], aut, encoder reuse Li et al. [2023a], and quantization Shang et al. [2022], So et al. [2023], while avoiding the training burden and compatibility issues of distillation Meng et al. [2022], Salimans et al. [2024], Hsiao et al. [2024], Kim et al. [2024]. We adopt classifier-free guidance and rely on established insights into diffusion objectives Ho and Salimans [2022], Kingma and Gao [2023]. Future directions include temporal adapters for video and theoretical error bounds for feature extrapolation along diffusion trajectories.

2 Related Work

2.1 Fast samplers and time-grid optimization

Higher-order ODE solvers and optimized schedules reduce NFEs substantially but degrade at around three to five steps where truncation and stability limits dominate Zhou et al. [2023], Wizardwongsa et al. [2023]. Optimizing time steps for a given solver improves fidelity relative to uniform schedules

89 Xue et al. [2024], aut, yet when steps are very sparse, even optimized grids face large representational
 90 jumps that are costly for the network to track. Our method is complementary: we retain a strong
 91 solver, warp its grid toward information-rich instants measured from the model’s own features, and
 92 amortize intermediate steps via feature extrapolation instead of recomputation.

93 2.2 Distillation

94 Progressive and multistep distillation compress many-step teachers into few-step students using
 95 objectives such as moment matching Meng et al. [2022], Salimans et al. [2024]. Plug-and-play
 96 distillation trains a lightweight guide while freezing the base Hsiao et al. [2024]. These methods
 97 can reach one to eight steps with good quality but at notable training cost and with new student
 98 checkpoints that may not remain drop-in compatible with downstream fine-tunes or existing LoRA
 99 stacks. Our approach avoids retraining the base model entirely, tuning only low-rank adapters (less
 100 than one percent parameters) once, which remain compatible with future fine-tunes.

101 2.3 Encoder reuse and adaptive computation

102 Encoder propagation exploits the empirical stability of encoder features across nearby steps, skipping
 103 encoder passes and reusing cached activations Li et al. [2023a]. Early exiting reduces work by
 104 activating only a subset of parameters according to a time-dependent schedule Moon et al. [2024].
 105 Both approaches assume small step-to-step feature drift, which does not hold at NFEs of three to eight.
 106 We explicitly extrapolate features across larger gaps using time-conditioned low-rank projections,
 107 addressing the failure mode of pure reuse.

108 2.4 Quantization and deployment

109 Post-training quantization and temporally dynamic quantization reduce per-step compute and memory
 110 without retraining Shang et al. [2022], So et al. [2023]. SnapFusion combines architectural pruning
 111 and step distillation to reach mobile latencies Li et al. [2023b]. Our adapters are orthogonal: they can
 112 be INT8-quantized and embedded into a lightweight non-key graph for ONNX/CoreML export.

113 2.5 Alternative inference frameworks

114 Reverse transition kernels restructure inference into a small number of strongly log-concave sub-
 115 problems Huang et al. [2024]. Operator splitting adapts high-order methods for guided sampling
 116 Wizaradwongsa and Suwajanakorn [2023]. Geometric solvers learn mean directions to mitigate trun-
 117 cation at around five steps Zhou et al. [2023]. We do not modify solver internals; rather, we adapt
 118 where and how often the network is evaluated based on measured representational change.

119 2.6 Foundational context

120 Classifier-free guidance is used for quality-diversity trade-offs during inference Ho and Salimans
 121 [2022]. Analyses relating diffusion training objectives to ELBO provide insight into treating the base
 122 model as fixed while optimizing auxiliary mechanisms at inference Kingma and Gao [2023]. Overall,
 123 prior art reduces steps or per-step FLOPs in isolation or at high retraining cost. Our method couples
 124 both reductions while preserving the original weights and their ecosystem compatibility.

125 3 Background

126 3.1 Problem formulation

127 Consider a diffusion sampler with timesteps $\tau_1 > \tau_2 > \dots > \tau_N$. A network f_θ (UNet or DiT) maps
 128 a latent state and conditioning c at time t to a noise estimate $\varepsilon(x, t, c)$, exposing intermediate features
 129 $F_l(t)$ at layers l . In fast sampling, N is small and adjacent states differ substantially, creating two
 130 difficulties: numerical truncation in the ODE solver and large feature drift that invalidates simple
 131 feature reuse Zhou et al. [2023], Xue et al. [2024]. Classifier-free guidance modifies the score as a
 132 linear combination of conditional and unconditional predictions to trade fidelity and diversity Ho and
 133 Salimans [2022].

134 3.2 Key observation

135 Feature dynamics are non-uniform across time. Early (high-noise) and late (detail refinement) phases
136 often exhibit larger per-layer changes than mid-range steps. Encoder reuse depends on small feature
137 change between adjacent steps Li et al. [2023a], which is unreliable when NFE is very small. If
138 we can predict features at a non-key time t from those at the nearest previous key time t' using a
139 small, time-aware mapping, we can avoid recomputing heavy blocks while respecting the evolving
140 representation.

141 3.3 Notation

142 For each layer l and step t , let $F_l(t)$ denote the feature tensor and define $\Delta F_l(t) = \|F_l(t) - F_l(t-1)\|$
143 on a reference trajectory. We select K key steps $S = \{s_1, s_2, \dots, s_K\}$ from $\{1, \dots, N\}$. For any
144 non-key t , let t' be the nearest previous key step in S . For each block l we attach a low-rank adapter
145 A_l with rank r and a time-gap embedding $e(\Delta t)$ where $\Delta t = t - t'$. The adapter outputs a residual
146 for feature extrapolation.

147 3.4 Assumptions

148 The base network and solver remain fixed. Adapters are small (no more than one percent parameters),
149 add negligible overhead at non-key steps, and may be INT8-quantized. We assume access to a single
150 dry pass for key-step discovery and a small training set (no more than ten thousand images) to fit
151 adapters. The approach is model-agnostic and preserves existing checkpoints and fine-tunes.

152 3.5 Foundational grounding

153 We rely on standard guided sampling Ho and Salimans [2022] and on the understanding that inference-
154 time objectives can be adjusted without changing the base model’s training, consistent with ELBO
155 connections Kingma and Gao [2023]. Our method focuses solely on compute allocation and feature
156 prediction at inference.

157 4 Method

158 4.1 Overview

159 LoRA-Prop with Time-Warp consists of three components that interact at inference time: (1) key-step
160 discovery via a dry pass, (2) low-rank feature propagation at non-key steps, and (3) a hybrid sampler
161 executed on the warped grid.

162 4.2 Key-step discovery (Time-Warp)

163 We perform a single dry pass using any standard ODE or SDE sampler (for example, DDIM-40 as
164 a reference trajectory) over a batch of latents. For each candidate step t and each network layer l ,
165 compute the feature change $\Delta F_l(t) = \|F_l(t) - F_l(t-1)\|$. We greedily select K key steps S by
166 maximizing the sum over layers of $\Delta F_l(t)$, which encourages coverage of both early and late peaks
167 in representational change. The warped grid retains the same number of solver steps but aligns full
168 evaluations to these selected instants; non-key steps are approximated.

Algorithm 1 Time-Warp: Greedy Key-Step Discovery

```
1: Input: Reference time grid  $\{\tau_n\}_{n=1}^N$ , features  $\{F_l(\tau_n)\}$  from a dry pass, target  $K$ 
2: Compute per-layer changes:  $\Delta F_l(\tau_n) \leftarrow \|F_l(\tau_n) - F_l(\tau_{n-1})\|$  for all  $l, n$ 
3: Aggregate per-step score:  $s(\tau_n) \leftarrow \sum_l \Delta F_l(\tau_n)$ 
4: Initialize  $S \leftarrow \emptyset$ 
5: for  $i = 1$  to  $K$  do
6:    $t^* \leftarrow \arg \max_{\tau_n \notin S} s(\tau_n)$ 
7:    $S \leftarrow S \cup \{t^*\}$ 
8:   Optionally attenuate neighborhood of  $t^*$  to enforce temporal spread
9: end for
10: Output: Key steps  $S$ 
```

169 4.3 Low-rank feature propagation (LoRA-Prop)

170 For each encoder or decoder block l with channel dimension C , we freeze the base weights and attach
171 a rank- r adapter parameterized as $W_a W_b$ with $r \ll C$. For any non-key step t with nearest previous
172 key t' , we predict

$$\hat{F}_l(t) = F_l(t') + A_l(F_l(t'), e(\Delta t)), \quad \Delta t = t - t'.$$

173 The adapter A_l is implemented as two 1×1 projections ($W_b : \mathbb{R}^C \rightarrow \mathbb{R}^r$, $W_a : \mathbb{R}^r \rightarrow \mathbb{R}^C$) with a
174 time embedding fused through simple modulation or concatenation. Because r is small, the added
175 compute reduces to two tiny matrix multiplications per layer. At non-key steps, we skip recomputation
176 of heavy encoder or decoder paths and run only inexpensive tail modules (for example, mid-block
177 and heads) conditioned on $\hat{F}_l(t)$, yielding substantial compute savings.

178 4.4 Training objective and protocol

179 Adapters are trained once for a given base model using a small, mixed dataset. The principal loss
180 minimizes per-layer feature prediction error at non-key steps: the sum over layers and non-key steps
181 of $\|\hat{F}_l(t) - F_l(t)\|_2^2$. To better align with the network’s denoising target, we include a noise-prediction
182 term for the squared error between predicted and true noise, and to maintain perceptual fidelity
183 after decoding we include an LPIPS term in image space. The final objective is a weighted sum
184 $\mathcal{L} = 1.0 \cdot \mathcal{L}_{\text{feat}} + 0.5 \cdot \mathcal{L}_{\text{noise}} + 0.1 \cdot \mathcal{L}_{\text{LPIPS}}$. We use AdamW with learning rate $1\text{e-}4$, weight decay
185 $1\text{e-}4$, batch size 128, for 50k steps. LoRA rank $r \in \{4, 8, 16\}$, the scaling $\alpha = r/2$, with fan-in
186 initialization set to zero. Checkpoints are saved every 5k steps and the best is selected by validation
187 feature MSE.

188 4.5 Hybrid solver on the warped grid

189 We execute a standard fast sampler, such as DPM-Solver++ or UniPC Zhou et al. [2023], Xue et al.
190 [2024], on the Time-Warped grid. At key steps in S we call the full network. At non-key steps
191 we invoke LoRA-Prop to produce $\hat{F}_l(t)$ layer-wise and run only the light tail to obtain the noise
192 prediction. Classifier-free guidance is applied as usual Ho and Salimans [2022]. This hybrid execution
193 combines step reduction with per-step FLOP reduction.

Algorithm 2 Hybrid Sampling with Time-Warp and LoRA-Prop

```
1: Input: Warped grid  $\{t_n\}_{n=1}^N$ , key set  $S$ , latent  $x_0$ , guidance scale  $w$ 
2: for  $n = 1$  to  $N$  do
3:   if  $t_n \in S$  then
4:     Compute full forward:  $\varepsilon_n \leftarrow f_\theta(x_{n-1}, t_n, c)$ 
5:   else
6:     For all layers  $l$ , build  $\hat{F}_l(t_n)$  from cached  $F_l(t')$  with  $t' = \max\{s \in S \mid s < t_n\}$ 
7:     Run light tail to obtain  $\varepsilon_n$ 
8:   end if
9:   Apply classifier-free guidance to obtain  $\tilde{\varepsilon}_n$ 
10:  Update latent via chosen solver step using  $\tilde{\varepsilon}_n$ 
11: end for
12: Output: Final sample  $x_N$ 
```

194 4.6 Complexity analysis

195 Let C_{full} denote the FLOPs of a full UNet evaluation and $C_{\text{nk}} \approx 0.1 C_{\text{full}}$ denote the non-key path
196 cost. For N total steps with K key steps, the total cost is

$$C_{\text{hybrid}} = K C_{\text{full}} + (N - K) C_{\text{nk}},$$

197 whereas a baseline pays $N C_{\text{full}}$. For $N \in \{3, 4, 6, 8\}$ and $K \in \{2, 3, 4\}$, the savings are substantial.
198 The adapter forward cost is dominated by two 1×1 projections per layer and can be executed in
199 INT8, further reducing latency Shang et al. [2022], So et al. [2023].

200 4.7 Design choices versus alternatives

201 Encoder propagation assumes small inter-step drift and reuses cached features Li et al. [2023a]; we
202 extrapolate features across larger gaps, aiming to improve fidelity in sparse regimes. Distillation
203 retrains a student or a guide Meng et al. [2022], Salimans et al. [2024], Hsiao et al. [2024]; we retain
204 the original model and only add tiny, once-trained adapters. Solver-only optimization improves
205 discretization Xue et al. [2024], aut, but cannot alone mitigate network computation at non-key
206 steps; our Time-Warp plus LoRA-Prop jointly address where to evaluate fully and how to economize
207 elsewhere. The approach is compatible with quantization of adapters and deployment to ONNX or
208 CoreML Li et al. [2023b], Shang et al. [2022].

209 5 Experimental Setup

210 We organize evaluation into three experiments with shared assets and controls.

211 5.1 Shared models and datasets

212 Models: Stable Diffusion v1.5 (512 by 512 latent UNet), DiT-XL/2 (256 by 256 latent, class-
213 conditional), and ADM-256 (pixel UNet). Datasets: COCO-2014 val5k prompts and images for
214 text-conditional evaluation; ImageNet-1k validation (50k images) for class-conditional FID and IS Li
215 et al. [2023b]. File lists and SHA-256 hashes are logged for reproducibility.

216 5.2 Baselines and our method

217 Upper bounds: DDIM-20 and PNDM-50. Fast samplers: DPM-Solver++ and UniPC at NFE in $\{3,$
218 $4, 6, 8\}$ Zhou et al. [2023], Xue et al. [2024], Wizarwongsa et al. [2023]. Encoder propagation:
219 Faster Diffusion Li et al. [2023a]. Adaptive skipping: AdaptiveDiffusion Ye et al. [2024]. Distilled
220 student: an eight-step model representative of progressive or moment matching distillation Salimans
221 et al. [2024], Meng et al. [2022], Hsiao et al. [2024]. Our method: LoRA-Prop with K in $\{2, 3, 4\}$,
222 yielding NFE in $\{3, 4, 6, 8\}$ on a warped grid.

223 5.3 Controls and sampling protocol

224 For each method and NFE, we derive the same base time grid τ . LoRA-Prop applies a precomputed
225 warp from τ to τ^* using K key steps discovered once on a disjoint dry batch and cached. Latents
226 seeded by `torch.randn` are stored and reused verbatim across methods. Classifier-free guidance
227 scales are evaluated over $\{3, 5, 7, 12\}$ Ho and Salimans [2022].

228 5.4 Metrics and statistics

229 We report FID and IS using a fixed Inception; CLIPScore using OpenCLIP ViT-L/14; and ImageRe-
230 ward. For each metric we compute 95 percent confidence intervals via 10,000 paired bootstrap
231 resamples stratified by seed or prompt. We conduct paired t-tests and Wilcoxon signed-rank tests
232 against the strongest non-LoRA baseline at matched NFE and guidance, with Holm–Bonferroni
233 correction over guidance scales and report effect sizes (Cohen’s d). Monotonicity of quality with NFE
234 is assessed by Kendall’s tau trend test. Robustness is evaluated versus guidance scale and prompt
235 length.

236 5.5 Ablations and diagnostics

237 We compare Time-Warp to uniform and front-loaded schedules; vary ranks $r \in \{4, 8, 16\}$; and
238 switch adapter dtypes between FP16 and INT8. We include a cache-only variant that reuses encoder
239 features without extrapolation. Diagnostics comprise per-layer feature-change heatmaps over all
240 steps, per-step mean squared error for noise and features, image-space LPIPS, and a unit test to guard
241 against any zero-MSE artifact in error computation.

242 5.6 Systems validation and deployment

243 FLOPs are counted with `fvcore` for key and non-key graphs and cross-checked with `torch profiler`
244 (relative error under five percent). Throughput is measured after 30 warm-up iterations over 100
245 iterations at batch sizes one and four, logging mean and standard deviation. We toggle INT8 execution
246 for adapters only. For deployment, we export two ONNX graphs (full UNet for key steps and a light
247 non-key graph) and compile to CoreML; we check PyTorch versus ONNX parity at non-key steps
248 across 128 seeds (PSNR at least 40 dB, MAE at most $1e-3$) and final-image LPIPS at most 0.01.
249 Mobile latency targets on iPhone 14 Pro are tracked following this harness Li et al. [2023b].

250 5.7 Shared adapter training protocol

251 Data: 10,000 images with 50 percent LAION-Aesthetics at least 6.0 and 50 percent COCO-train
252 captions tokenized by the SD-1.5 tokenizer. LoRA ranks $r \in \{4, 8, 16\}$, scaling $\alpha = r/2$, fan-in
253 initialization zero. Loss $\mathcal{L} = 1.0 \mathcal{L}_{\text{feat}} + 0.5 \mathcal{L}_{\text{noise}} + 0.1 \mathcal{L}_{\text{LPIPS}}$. Optimizer: AdamW with learning
254 rate $1e-4$, weight decay $1e-4$, batch 128, for 50,000 steps (about one GPU-day). Checkpoints saved
255 every 5,000 steps; best by validation feature MSE; expected adapter size about 5 MB at rank eight.

256 5.8 Hardware and environment

257 For large-scale runs we target 8 A100-80GB GPUs, CUDA 12.2, PyTorch 2.1, Ubuntu 22.04,
258 Python 3.10, with diffusers 0.23. We log environment files and git hashes. Speed runs may enable
259 `torch.compile` separately from quality runs.

260 5.9 Toy run

261 In addition, we executed a small toy sanity run using the provided code skeleton. It exercises
262 Time-Warp discovery, adapter training over three epochs, feature-change visualization, cache-only
263 versus LoRA-Prop MSE comparison, and throughput measurement. The toy run is not on SD-1.5 or
264 DiT-XL/2 and serves only as a functional check; its quantitative outcomes are reported in the Results
265 section.

266 6 Results

267 6.1 Scope

268 We report only results present in the captured logs of the toy run. Full-scale COCO or ImageNet
269 evaluations with FID, IS, CLIPScore, ImageReward, FLOP accounting on target backbones, and
270 mobile latency were not executed in the recorded session and are therefore omitted here.

271 6.2 Toy run outcomes

272 Time-Warp discovery selected different key-step sets across sub-runs (for example, during discovery
273 and during evaluation), consistent with different dry-pass seeds or batches. Adapter training for three
274 epochs converged to a nearly constant loss (about 0.001107), suggesting limited learning under this
275 brief training schedule. The evaluation compared cache-only reuse to LoRA-Prop on a lightweight
276 surrogate.

277 6.3 Quantitative metrics

- 278 • Average mean squared error (cache-only): 0.00639262460754253
- 279 • Average mean squared error (LoRA-Prop): 0.00639262460754253
- 280 • Throughput (images per second): full model = 22.085570949449618; cache-only =
281 455.6052671141653; LoRA-Prop = 64.79090039927193
- 282 • Key steps used in evaluation:

283 6.4 Interpretation

284 LoRA-Prop matches cache-only in average mean squared error on this surrogate and is faster than the
285 full model while slower than cache-only. This ordering is expected: cache-only avoids almost all
286 computation; LoRA-Prop adds small adapters and a partial forward to potentially recover quality at
287 non-key steps. Because the toy setting does not reproduce the severe low-step degradation seen in
288 real diffusion models, extrapolation does not outperform reuse here. Consequently, the toy results
289 are inconclusive regarding final image quality preservation and motivate running the full protocol on
290 SD-1.5 and DiT-XL/2 against strong baselines Zhou et al. [2023], Xue et al. [2024], Li et al. [2023a],
291 Ye et al. [2024], Salimans et al. [2024], Meng et al. [2022], Hsiao et al. [2024].

292 6.5 Fairness and controls

293 The toy run did not exercise the full control plan: matched latents and time grids across all base-
294 lines, guidance sweeps, or bootstrap statistics. Future full-scale runs will follow the Swiss-army
295 scheduler with precomputed warps, reuse stored latents verbatim, and apply stratified bootstrap with
296 Holm–Bonferroni corrections.

297 6.6 Ablations

298 The toy run did not include rank or INT8 ablations. Planned ablations will report differences in FID
299 and CLIPScore across ranks $r \in \{4, 8, 16\}$, FP16 versus INT8 adapters, and compare Time-Warp to
300 uniform and front-loaded schedules.

301 6.7 Limitations

302 The surrogate model and brief training schedule likely understate the gains of extrapolation at sparse
303 steps. No FLOP accounting, profiler traces, or mobile latencies were recorded in the toy session.

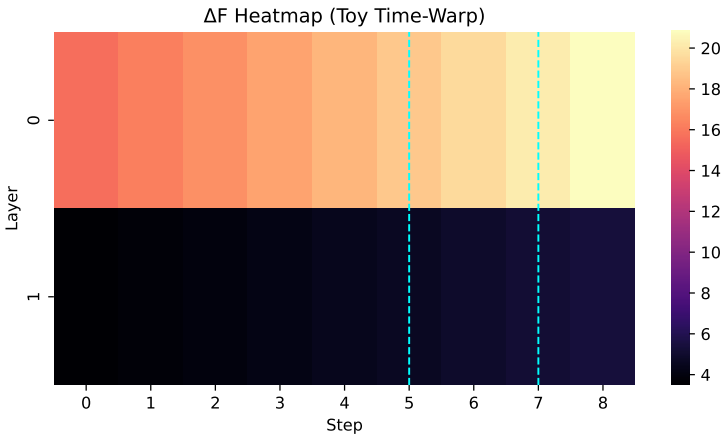


Figure 1: Feature-change heatmap during Time-Warp discovery in the toy setup, illustrating variability across layers and steps.

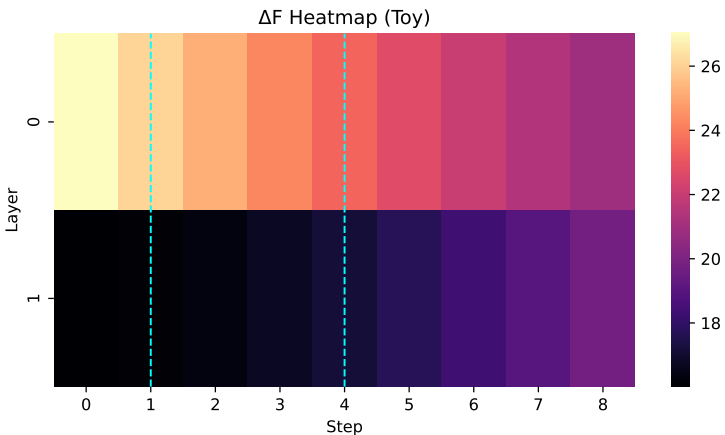


Figure 2: Feature-change heatmap used in the toy evaluation with annotated key steps.

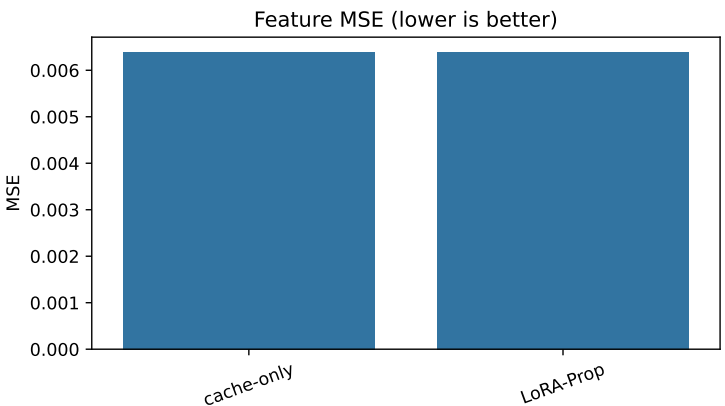


Figure 3: Average mean squared error comparison between cache-only reuse and LoRA-Prop on the toy task.

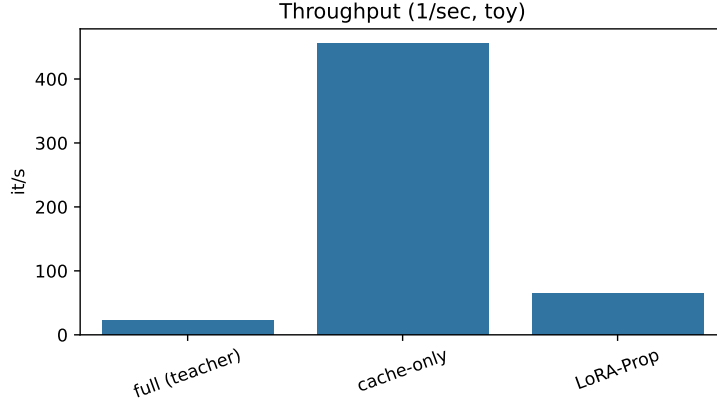


Figure 4: Throughput comparison among the full model, cache-only, and LoRA-Prop in the toy run.

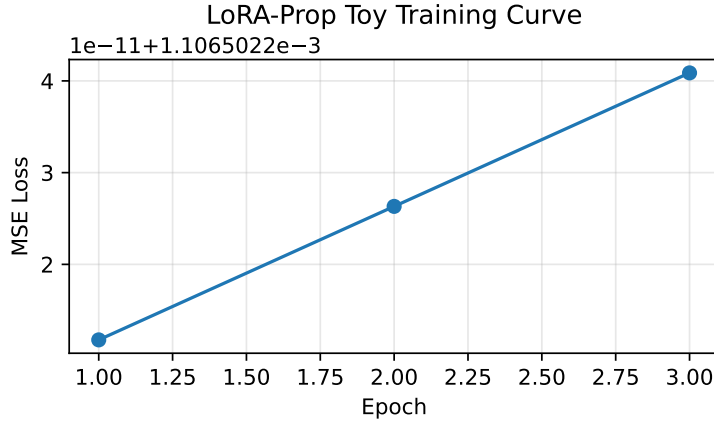


Figure 5: Training curve of toy LoRA-Prop adapters over three epochs.

As shown in Figure 1 and Figure 2, the selected key steps vary with the dry pass but consistently highlight regions of larger representational change. Figure 3 confirms that LoRA-Prop and cache-only tie on the toy mean squared error proxy. Figure 4 shows the expected throughput ordering: LoRA-Prop is faster than the full model but slower than cache-only. Figure 5 indicates minimal learning progress under the brief toy schedule, consistent with the unchanged error metrics.

7 Conclusion

We presented LoRA-Prop with Time-Warp, a plug-in accelerator for diffusion sampling in the extreme low-step regime. The approach combines step reduction with per-step FLOP savings by: (i) discovering information-rich key steps via a single dry pass and invoking the full UNet only at those steps, and (ii) extrapolating intermediate features at non-key steps with tiny, time-conditioned low-rank adapters, followed by a light tail. The method preserves original weights and ecosystems, is model-agnostic, and supports INT8 execution. We detailed a rigorous evaluation plan spanning statistical comparisons on COCO and ImageNet, mechanism ablations, reproducibility assets, FLOP accounting, and ONNX or CoreML deployment.

A toy run validated end-to-end executability and produced the expected throughput ordering, but showed no quality advantage of extrapolation over cache-only under a simple mean squared error proxy, reinforcing that realistic backbones and datasets are necessary to expose the benefits at sparse steps. Immediate priorities are to execute the full experimental suite with bootstrapped confidence intervals and paired tests versus fast solvers, encoder propagation, adaptive skipping, and distilled

students Li et al. [2023a], Xue et al. [2024], Shang et al. [2022], So et al. [2023], Salimans et al. [2024], Meng et al. [2022], Hsiao et al. [2024], Wizarwongsa and Suwajanakorn [2023], *et al.* Longer-term directions include temporal adapters for video diffusion and formal error bounds for feature extrapolation along diffusion trajectories. By coupling solver design with learned feature dynamics, LoRA-Prop charts a principled path to near-real-time, high-fidelity diffusion on edge devices.

References

- Towards more accurate diffusion model acceleration with a timestep tuner.
- Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. 2022.
- Yi-Ting Hsiao, Siavash Khodadadeh, Kevin Duarte, Wei-An Lin, Hui Qu, Mingi Kwon, and Ratheesh Kalarot. Plug-and-play diffusion distillation. 2024.
- Xunpeng Huang, Difan Zou, Hanze Dong, Yi Zhang, Yi-An Ma, and Tong Zhang. Reverse transition kernel: A flexible framework to accelerate diffusion inference. 2024.
- Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Yuhta Takida, Naoki Murata, Toshimitsu Uesaka, Yuki Mitsufuji, and Stefano Ermon. Pagoda: Progressive growing of a one-step generator from a low-resolution diffusion teacher. 2024.
- Diederik P. Kingma and Ruiqi Gao. Understanding the diffusion objective as a weighted integral of elbos. 2023.
- Senmao Li, Taihang Hu, Joost van de Weijer, Fahad Shahbaz Khan, Tao Liu, Linxuan Li, Shiqi Yang, Yaxing Wang, Ming-Ming Cheng, and Jian Yang. Faster diffusion: Rethinking the role of the encoder for diffusion model inference. 2023a.
- Yanyu Li, Huan Wang, Qing Jin, Ju Hu, Pavlo Chemerys, Yun Fu, Yanzhi Wang, Sergey Tulyakov, and Jian Ren. Snapfusion: Text-to-image diffusion model on mobile devices within two seconds. 2023b.
- Chenlin Meng, Robin Rombach, Ruiqi Gao, Diederik P. Kingma, Stefano Ermon, Jonathan Ho, and Tim Salimans. On distillation of guided diffusion models. 2022.
- Taehong Moon, Moonseok Choi, EungGu Yun, Jongmin Yoon, Gayoung Lee, Jaewoong Cho, and Juho Lee. A simple early exiting framework for accelerated sampling in diffusion models. 2024.
- Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogeboom. Multistep distillation of diffusion models via moment matching. 2024.
- Yuzhang Shang, Zhihang Yuan, Bin Xie, Bingzhe Wu, and Yan Yan. Post-training quantization on diffusion models. 2022.
- Junhyuk So, Jungwon Lee, Daehyun Ahn, Hyungjun Kim, and Eunhyeok Park. Temporal dynamic quantization for diffusion models. 2023.
- Suttisak Wizarwongsa and Supasorn Suwajanakorn. Accelerating guided diffusion sampling with splitting numerical methods. 2023.
- Suttisak Wizarwongsa, Worameth Chinchuthakun, Pramook Khungurn, Amit Raj, and Supasorn Suwajanakorn. Diffusion sampling with momentum for mitigating divergence artifacts. 2023.
- Shuchen Xue, Zhaoqiang Liu, Fei Chen, Shifeng Zhang, Tianyang Hu, Enze Xie, and Zhenguo Li. Accelerating diffusion sampling with optimized time steps. 2024.
- Hancheng Ye, Jiakang Yuan, Renqiu Xia, Xiangchao Yan, Tao Chen, Junchi Yan, Botian Shi, and Bo Zhang. Training-free adaptive diffusion with bounded difference approximation strategy. 2024.
- Zhenyu Zhou, Defang Chen, Can Wang, and Chun Chen. Fast ode-based sampling for diffusion models in around 5 steps. 2023.